

Embedding Inferred JML Contracts in Java Bytecode and OpenAPI Specifications

Brendan Edmonds, Mark Utting

UQ Cyber, School of Electrical Engineering and Computer Science
The University of Queensland, St Lucia QLD 4067, Australia
{b.edmonds, m.utting}@uq.edu.au

Abstract—Consumers of compiled Java libraries and remote REST services rarely have the source code that produced them. This makes automatically inferred Java Modeling Language (JML) specifications, which have been shown to materially improve downstream tooling, unavailable to the very consumers that would benefit from them: test generators, IDEs, verifiers, and large language models reading APIs at runtime. Existing channels for shipping specifications alongside compiled artefacts — JML stub files, sidecar files, runtime assertions — are either ignored by the standard toolchain or restricted to a narrow set of properties. We propose a uniform mechanism for distributing inferred JML contracts in both settings: a runtime-retained Java annotation, `@JmlSpec`, written into class-file attributes by an ASM-based embedder, and a parallel OpenAPI 3.x extension family (`x-jml-*`) for service boundaries. We evaluate the embedder on three real-world Java libraries (Apache Commons Lang, Apache Commons IO, and Guava) totalling 20,546 methods with synthetic specifications that isolate the embedding mechanism, and on a real-inferred-specs corpus of 73 methods that exercises the full inference → embedding pipeline. We measure 100.0% roundtrip fidelity across all four corpora, 11.76–13.49% bytecode-size overhead on the synthetic-spec runs, embedding throughput in the 10,000–31,000 methods/second range, and reading throughput in the 134,000–316,000 methods/second range (medians 18,859–29,358 and 162,002–275,834 methods/second respectively, across 39 runs). A negative test confirms that classes embedded with `@JmlSpec` load correctly in Java Virtual Machines (JVMs) that have no awareness of the annotation type. The contribution is a path from inferred specifications to consumer-readable contracts that requires no changes to the build toolchain: any existing JAR can be enriched in place.

Index Terms—JML, Java bytecode, ASM, OpenAPI, specification distribution, runtime annotations, software contracts

I. INTRODUCTION

Recent work has shown that automatically inferred Java Modeling Language (JML) specifications materially improve LLM-generated unit tests on real-world Java code [1]. That result, and others like it on inferred specifications for verification and tooling, share an unstated assumption: the consumer of the specifications — the test generator, the IDE, the verifier, the language model reading the API at inference time — has access to the same source tree the inferrer ran over. The assumption is rarely satisfied in practice. Java libraries are distributed as JAR files; REST endpoints expose only their

HTTP signature; gRPC services expose only their protobuf descriptors. Source code is the exception, not the norm.

The conceptual question of whether specifications *should* be transported alongside compiled artefacts has a long history [2]–[4]. The practical question is whether the standard toolchain can be made to carry them without modification. In this paper, we propose and evaluate a uniform mechanism for distributing inferred JML contracts in compiled Java libraries and in REST service interfaces: a runtime-retained annotation written into class-file attributes by an ASM-based embedder, plus a parallel OpenAPI 3.x extension family for service boundaries.

We address the following research questions:

- **RQ1.** Can inferred JML contracts be embedded in compiled Java artefacts and reproduced with full fidelity from the embedded bytes, with no source-code dependency?
- **RQ2.** What is the bytecode-size overhead of the embedding, and how does it scale across libraries of different shapes (utility code, I/O abstractions, generics-heavy collections)?
- **RQ3.** Is the embedding and reading throughput sufficient to support per-pull-request continuous-integration workflows on production-scale libraries?
- **RQ4.** Does an OpenAPI extension family integrate with existing OpenAPI tooling (parsers, generators) without modification?

We evaluate the bytecode side empirically on Apache Commons Lang, Apache Commons IO, and Guava (20,546 methods total) and the OpenAPI side by construction against the OpenAPI 3.x specification and the standard parsers (swagger-parser, openapi-generator). We summarise the principal findings in Table I and the supporting analysis in Section V. In short: roundtrip fidelity is 100.0% on all three libraries; bytecode overhead sits at 11.76–13.49%; embedding throughput ranges from 10,194 to 31,095 methods/second (median 22,198 for the largest library, Guava); reading throughput is roughly an order of magnitude higher; and the OpenAPI extension is preserved by both standard parsers without modification.

Contributions. In summary, this paper makes the following contributions:

- A Java annotation type, `@JmlSpec`, repeatable per clause, written by an ASM-based embedder into class-file `RuntimeVisibleAnnotations` attributes, from which reflection or any class-file reader recovers the embedded specification.
- An OpenAPI 3.x extension family (`x-jml-requires`, `x-jml-ensures`, `x-jml-assignable`, `x-jml-signals`, `x-jml-invariant`) carried per-operation and per-schema, preserved by standard OpenAPI parsers without modification.
- A Maven classifier sidecar (`<artifact>-jml.jar`) carrying JML stub files, used when in-bytecode embedding is unsuitable (signed JARs that cannot be re-signed; specifications too large to fit the per-class budget).
- An empirical evaluation across 20,546 methods of three real-world Java libraries showing 100.0% roundtrip fidelity, 11.76–13.49% bytecode-size overhead, and embedding/reading throughput high enough to support per-pull-request continuous-integration workflows.
- A negative test confirming that classes embedded with `@JmlSpec` load correctly in Java Virtual Machines that have no awareness of the annotation type.

Paper organisation. The remainder of the paper is structured as follows. Section II reviews the existing channels for shipping specifications alongside compiled Java code and the gap each of them leaves. Section III presents the format design — the annotation type, the canonical-form printer, and the OpenAPI extension schema. Section IV describes the implementation. Section V reports the empirical validation. Section VI discusses generalisability; Section VII the threats to validity. Section VIII surveys related work and Section IX concludes.

II. BACKGROUND AND MOTIVATION

Source-level JML specifications are well established: the language [3], [4] and its checker [5] have a thirty-year history, and recent inference work [1] produces them at scale. The gap we address is downstream of inference: once a specification is known for a method, how does it travel to the consumer that needs it?

Three transport mechanisms predate this work, none of them sufficient on its own.

a) JML stub files. Files of the form `<class>.jml` or `<class>.refines-jml` carry signatures and clauses without bodies. OpenJML reads stubs natively; nothing else in the standard Java ecosystem does. A stub file must travel separately from the JAR (a sibling artefact, a Maven classifier, or a path entry). For projects that already use OpenJML, stubs are the right artefact; for projects that consume specifications from anywhere else — IDEs, test generators, runtime checkers — stubs are invisible.

b) Annotations. The Java annotation mechanism (JLS §9.6) is the standard channel for metadata, but the existing annotations cover only fragments of

what specifications need. JSR-305 captured nullability (`@Nullable`, `@NotNull`) and was withdrawn before final standardisation; the surviving informal usage covers only nullability. The Checker Framework’s annotations are type-system-shaped and do not generalise to `\sum`, `\forall`, or relational postconditions. JSR-308’s pluggable type system extended where annotations may appear but did not extend what they may say. The result is a partial channel: annotations survive the toolchain, but the existing vocabulary is too thin.

c) Class-file attributes. JVM §4.7.1 permits vendor-defined class-file attributes. A custom attribute could carry arbitrary structured content, and class loaders ignore unrecognised attributes. The cost is invisibility: standard reflection does not surface custom attributes, IDEs do not display them, ProGuard/R8 strip them by default, and javap requires `-private -v` to even mention them. A custom attribute is a write-only channel for any consumer that does not adopt a custom reader.

d) REST API descriptions. At service boundaries, OpenAPI 3.x and gRPC’s protobuf carry a complementary class of metadata. OpenAPI permits arbitrary `x-*` extension properties; gRPC has no comparable extension point but supports custom Protobuf annotations. Neither standard says anything about preconditions or postconditions; they describe the wire format, not the contract.

e) The carrier we propose. A repeated runtime-retained annotation type (`@JmlSpec`, with one annotation per JML clause) plus an OpenAPI extension family (`x-jml-*`) covers the gap. The annotation is read by reflection, by ASM, and by javap; the OpenAPI extension is preserved by the standard parsers; both survive the Maven and Gradle ecosystems unmodified. Section III specifies the format precisely; Section IV describes our embedder/extractor implementation; Section V reports the empirical evaluation.

III. FORMAT DESIGN

The format design is the contract between the embedder and any future consumer. It must be precise enough that an alternative embedder or reader can be written from the specification alone.

A. The `@JmlSpec` Annotation Type

```
package com.jml.spec;

@Repeatable(JmlSpecs.class)
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.METHOD, ElementType.CONSTRUCTOR,
        ElementType.FIELD, ElementType.TYPE})
public @interface JmlSpec {
    Kind kind();
    String text();
    int order() default 0;
    String version() default "1.0";
    String targetSignature() default "";
}

public enum Kind {
    REQUIRES, ENSURES, ASSIGNABLE, SIGNALS,
    LOOP_INVARIANT, LOOP_DECREASES, INVARIANT,
}
```

```
ALSO_PARTITION, AXIOM, INITIALLY
```

```
}
```

The choice of one annotation per clause (rather than a single annotation with all clauses as element values) is motivated by two needs. First, the order of `requires` clauses is load-bearing for well-definedness chains in JML: a precondition that asserts an array element’s value depends on earlier preconditions establishing the array’s non-null and the index’s in-bounds. The `order` element preserves source order on the wire. Second, forward compatibility: a consumer parameterised by a maximum supported `Kind` can ignore unknown kinds without skipping the rest of the annotations on the same method. A struct-shaped annotation would break consumers as new kinds were added.

The `targetSignature` element resolves synthetic carriers. Lambdas, method references, anonymous inner classes, and record accessors compile to synthetic methods or classes; the embedder writes specifications to the synthetic carrier with `targetSignature` pointing back to the user-meaningful signature.

B. Canonical Form

To support roundtrip equality at the byte level, clause text is canonicalised before emission. Whitespace runs collapse to single spaces; no spaces around `.` or `::`; single space around binary operators (`+`, `-`, `*`, `/`, `%`, comparison, logical); no space inside parentheses or brackets; `=>` and `<=>` are normalised to the long forms `==>` and `<==>`. Quantifier headers retain their bound-variable names; alpha-renaming is deferred to a future release and currently a no-op.

The canonicaliser is explicitly lexical: it does not perform semantic equivalence. Two clauses that differ in operand order ($a > b$ vs $b < a$) remain distinct strings, and roundtrip equality is preserved on the wire form rather than on a normalised theory level. Semantic equivalence would silently rewrite valid specifications during transit, which is unsuitable when the embedder’s job is to carry data faithfully.

C. The OpenAPI Extension

```
paths:
  /orders/{id}:
    get:
      x-jml-requires:
        - "id != null"
        - "id > 0"
      x-jml-ensures:
        - "\\result != null ==> \\result.id == id"
      x-jml-assignable:
        - "\\nothing"
      x-jml-signals:
        - exception: NotFoundException
          condition: "id > 0 && !exists(id)"
```

Each extension key is an array of canonical-form strings. `x-jml-requires` preserves order; the others are unordered. `x-jml-signals` is an array of `{exception, condition}` objects; exception types are JVM internal names so cross-language consumers can resolve them.

A JSON Schema document accompanies the artefact and may be used to validate any OpenAPI document independently of the inferer. When the OpenAPI document is generated by the framework (springdoc-openapi, JAX-RS) and not editable, a sidecar JSON file (`<service>.contract.json`) carries the same content keyed by route and verb.

D. Sidecar JAR

For specifications that exceed the per-class bytecode budget (rare in practice but possible when a method has many quantified loop invariants), and for signed JARs that cannot be re-signed, the embedder emits a parallel artefact with Maven classifier `-jml`. The sidecar contains JML stub files (`<class>.jmlspec`) keyed by class internal name and a manifest entry recording the source JAR’s SHA-256 hash so consumers can detect drift. Stub files are the format OpenJML reads natively, so the sidecar is consumable by the verifier without further transformation.

IV. IMPLEMENTATION

The embedder is implemented in Java 21, using ASM 9.7 for class-file rewriting. The module `com.jml:jml-embedder:1.0.0` is approximately 700 lines of code and depends only on ASM and SLF4J. Three main classes carry the production logic.

A. AsmJmlSpecWriter

The writer accepts either a single class file (as `byte[]`) or a JAR (as `Path`). For a JAR, it walks each `.class` entry, applies a `ClassReader` $\rightarrow$ `ClassVisitor` $\rightarrow$ `ClassWriter` pipeline that intercepts `visitMethod` calls, and emits a `@JmlSpecs` container annotation with one inner `@JmlSpec` per clause. Order is preserved via the `order` element. Methods without a corresponding `MethodSpec` are passed through unchanged.

For specifications that exceed the per-class bytecode budget or for JARs that must remain unmodified, the writer also produces a sidecar JAR per Section III. The sidecar’s JML stub files are generated by a small printer that decodes JVM method descriptors back into source-level types (with array dimensions, parameterised generic erasure, and primitive types).

B. AsmJmlSpecReader

The reader is symmetric: it walks each `.class` entry with a `ClassReader` configured to skip code, debug, and frames (it needs only the annotations) and accumulates a `Map<MethodKey, MethodSpec>`. The reader tolerates both the `@JmlSpecs` container shape (the canonical write form) and standalone repeated `@JmlSpec` annotations (the legacy shape that earlier annotation processors emit). Per-kind clauses are sorted by `order` on read.

C. JmlCanonicaliser

The canonicaliser implements the rules of Section III. It tokenises the clause text using a small lexer (identifiers, numeric literals, multi-character operators, JML backslash-keywords) and re-emits the tokens with canonical spacing. The implementation is approximately 250 lines and validates with 16 test cases covering whitespace collapsing, operator spelling, parenthesisation, and idempotency.

D. Pipeline Integration

The inferer integrates the embedder via a small bridge class, `InfererSpecConverter`, that translates the inferer’s `MethodSpecification` model to the embedder’s `MethodSpec` record. The conversion is byte-for-byte exact and preserves clause-list order; canonicalisation is invoked separately when needed. With the converter in place, the end-to-end pipeline is one method call:

```
MethodSpecification inferred = new
    MethodSpecificationInferer()
        .inferSpecification(method);
MethodSpec embeddable =
    InfererSpecConverter.toEmbeddable(inferred);
new AsmJmlSpecWriter().embedJar(input, output,
    Map.of(methodKey, embeddable));
```

The roundtrip property `extract(embed(infer(source))) == infer(source)` is exercised by a JUnit suite (`InfererToEmbedderRoundtripTest`) at small scale and at corpus scale by the validation harness of Section V.

V. EMPIRICAL VALIDATION

The embedder is evaluated on real-world Java libraries. The harness applies the embedder to each library’s JAR file, generates a synthetic specification per method (one requires clause per non-primitive parameter, one ensures clause for reference returns, and assignable `\nothing`), and measures four quantities: roundtrip fidelity, bytecode-size overhead, embedding throughput, and reading throughput.

A. Subjects

Three libraries are reported: Apache Commons Lang 3.14.0, Apache Commons IO 2.13.0, and Guava 33.3.0-jre. Commons Lang is the same subject used in Article 1; Commons IO contributes a different shape (file/stream utility classes with significant abstract-method counts); Guava is selected as a larger contrast with more complex use of generics and a multi-release JAR. Together the three libraries cover 20,546 methods. The remaining seven libraries identified in the protocol (Apache Commons Math, jOOL, Vavr, jsoup, JFreeChart, JUnit, AssertJ) are deferred to a follow-up validation; the validation harness fires uniformly across the three reported libraries, so the additional libraries are mechanical to validate.

B. Method

For each library:

- 1) Walk every `.class` entry; collect method signatures, excluding bridge, synthetic, and class-initialiser methods.
- 2) Generate a synthetic `MethodSpec` per method as above.
- 3) Embed the spec map into a copy of the JAR using `AsmJmlSpecWriter`, recording wall time.
- 4) Read the embedded JAR back using `AsmJmlSpecReader`, recording wall time and matching specs against the originals on (`internalName`, `methodName`, `descriptor`) keys.
- 5) Compute fidelity (matched / total), byte overhead (embedded / original – 1), and throughput (methods / wall time).

A separate negative test loads an embedded class via a `URLClassLoader` rooted only at the embedded JAR (with the platform class loader as parent) and confirms that `Class.forName` succeeds. The annotation type `com.jml.spec.JmlSpec` is not on this loader’s classpath; reflective annotation access is allowed to raise `TypeNotPresentException` but class load itself must not fail.

C. Results

Table I reports the headline measurements. Method counts, fidelity, and bytecode-size overhead are deterministic; we report each exactly once. Embedding and reading throughput vary across runs (JIT warm-up, garbage collection, OS scheduling); we report the median across 39 independent runs of the harness per library, with the observed ranges discussed in the Throughput paragraph below.

TABLE I
EMBEDDER VALIDATION ON REAL-WORLD LIBRARIES (MEDIAN ACROSS 39 RUNS FOR THROUGHPUT; FIDELITY AND OVERHEAD ARE DETERMINISTIC). THE FOURTH ROW REPORTS THE FULL INFERER→EMBEDDER PIPELINE ON REAL INFERER-GENERATED SPECIFICATIONS; THE SYNTHETIC SPECS IN THE FIRST THREE ROWS ARE DECOUPLED FROM THE INFERER’S BEHAVIOUR TO ISOLATE THE EMBEDDING MECHANISM. ON MATCHED METHODS, FIDELITY EQUALITY IS EXACT (ZERO CLAUSE-LEVEL MISMATCHES).

Library	Methods	Fidelity	Byte ovhd.	Embed (m/s)
<i>Synthetic specs (isolating the embedding mechanism)</i>				
Apache Commons Lang 3.14.0	4,029	100.0%	+11.95%	29,358
Apache Commons IO 2.13.0	2,677	100.0%	+13.49%	18,859
Guava 33.3.0-jre	13,840	100.0%	+11.76%	22,198
<i>Real inferred specs (full inferer→embedder pipeline)</i>				
JML-Inferer self-test	73	100.0%	—	—

D. Analysis

a) *Fidelity.*: All three libraries achieve 100.0% roundtrip fidelity. An earlier prototype of the embedder reported 96.9% / 97.6% fidelity on Commons Lang and Guava owing

to a missed callback in the writer’s `MethodVisitor` lifecycle: lazy emission was triggered by `visitCode`, `visitParameter`, `visitAnnotationDefault`, or `visitAnnotation`, but none of these hooks fires for abstract / interface methods, which only invoke `visitEnd`. Adding `visitEnd` to the trigger set closes the gap. Every method on every interface in all three libraries now receives the embedded annotation. Commons IO has a noticeably higher proportion of interfaces (its byte-overhead figure of 13.5% is correspondingly higher than Lang and Guava, which sit at 11.8–12.0%), confirming that the previous prototype would have shown the largest gap on Commons IO.

b) Byte overhead.: The 11.76–13.49% overhead reflects a worst-case workload: the synthetic specs emit one annotation per parameter for every method, regardless of whether a real inferer would have produced one. Real inferred specifications are typically smaller per method (preconditions are often empty for simple getters; assignable \nothing is a frequent, short, single-clause annotation). The absolute overhead is 77 KB for Commons Lang, 64 KB for Commons IO, and 354 KB for Guava — comfortable compared to typical Maven distribution sizes. The differences across libraries are explained by structural shape rather than total size: Commons IO has a higher proportion of interfaces (each of which receives a clause set per method), pushing its overhead percentage above the other two. For libraries where the cost is unacceptable, the Maven classifier sidecar (Section III) is the recommended fallback.

c) Throughput.: Across 39 runs of the harness, embedding throughput ranges from 10,194 to 31,095 methods/second (medians: 18,859 for Commons IO, 22,198 for Guava, 29,358 for Commons Lang); reading throughput ranges from 133,683 to 315,693 methods/second (medians: 162,002, 205,430, 275,834 respectively). At the medians, embedding a 14,000-method JAR completes in roughly 0.6 seconds end-to-end; reading completes an order of magnitude faster (the reader skips full class rewriting). These figures comfortably support per-PR-comment workflows in continuous-integration pipelines: even a pessimistic re-run on every push of a Guava-sized JAR is sub-second, well below the typical Maven build time the inference pipeline rides on.

d) Negative test.: `org.apache.commons.lang3.StringUtils` loaded successfully via a `URLClassLoader` that does not see `com.jml.spec.JmlSpec`. Reflective annotation access either returns the resolvable annotations (none in this loader’s view) or raises `TypeNotPresentException`, both documented behaviours. Class load is unaffected.

E. Real-inferred-specs roundtrip

The synthetic-spec validation isolates the embedding mechanism from the inferer’s behaviour, but does not exercise the inferer-to-embedder pipeline at scale. To close that gap a second harness, `CorpusLevelRoundtripTest`, runs the inferer over the inferer’s own model, visitor, and processor packages (4 source files, 73 non-trivial inferred specifications),

embeds the resulting `MethodSpec` entries into the inferer’s packaged JAR via `AsmJmlSpecWriter`, reads them back via `AsmJmlSpecReader`, and checks clause-list equality on the `MethodSpec` records.

Result: 73 of 73 methods (100%) roundtrip with byte-for-byte equality on every clause list. The full pipeline source → `MethodSpecificationInferer` → `InfererSpecConverter` → `AsmJmlSpecWriter` → bytecode → `AsmJmlSpecReader` → `MethodSpec` preserves every inferred clause exactly.

F. Threats specific to this measurement

- *Three libraries plus self-test*, not the protocol’s full ten external libraries. The fidelity analysis above identifies the residual risks (interface methods, multi-release JARs); on libraries that exercise either pattern the embedder reaches 100%. The follow-up validation against the remaining seven libraries (Apache Commons Math, jOOL, Vavr, jsoup, JFreeChart, JUnit, AssertJ) is mechanical.
- *JDK 21 only*. Older JDKs (8, 11, 17) and ahead-of-time compilers (GaalVM `native-image`) are deferred. The class files emitted by Commons Lang target Java 8, so the runtime-side validation should generalise without modification.
- *Real-inferred-specs sample is the inferer’s own model+visitor+processor packages*. The packages are non-trivial (POJOs with derived state, AST visitors, file-system orchestration) but cover only the data-model and orchestration tier of the inferer; the analyzer tier (16 000 LOC of analysis code) is excluded from the corpus to keep the test fast. Extension to the full inferer codebase, and to the published Apache Commons Lang sources, is mechanical.

VI. DISCUSSION

A. Why an annotation, not a class-file attribute?

Custom class-file attributes (JVMS §4.7.1) are a tempting alternative: they avoid the constant-pool pollution of large string literals and they are technically the more general mechanism. Three reasons argue against them in practice:

- *Tool visibility*. Standard reflection ignores custom attributes. IDEs do not display them. ProGuard and R8 strip unrecognised attributes by default unless explicitly listed. Each downstream consumer would need a custom reader.
- *Repeatable annotations are no longer onerous*. Java 8’s repeatable-annotation mechanism makes per-clause emission straightforward. The historical reason to prefer a custom attribute — avoiding one annotation per clause being emitted with a verbose container — is no longer compelling.
- *Forward compatibility*. An annotation-based carrier with a `Kind` enum allows new clause kinds to be added without breaking existing readers. A custom attribute’s binary layout is harder to evolve compatibly.

The annotation carrier is therefore the recommended primary form. The custom-attribute alternative remains usable for cases where the annotation is unsuitable (signed JARs, aggressive obfuscation), and the sidecar JAR fills the same gap with even less coupling to the bytecode.

B. Generality

The format design is independent of the inferer that produces the specifications. Any source of JML clauses — the inferer of Article 1, Daikon [6], hand-written contracts, contracts produced by a future LLM-assisted refinement step — can be embedded by the same mechanism. The format also generalises to languages other than Java that target the JVM (Kotlin, Scala) at the cost of those languages providing their own translators between source-level annotations and the bytecode `@JmlSpec` representation.

For non-JVM languages, the OpenAPI extension carries equivalent information at the service boundary. The bytecode side is JVM-specific; the service-boundary side is language-agnostic.

C. Limitations

The embedder is consumer-tolerant but the consumer has work to do. A reader that wants to act on the embedded clauses must (a) parse the canonical-form JML strings, (b) substitute parameter names, and (c) decide what to do with the result. We ship the writer, the reader, and the canonicaliser, but we do not ship a verifier or a runtime checker — those are downstream concerns that benefit from the work here without being implemented by it.

The 3% fidelity gap reported in Section V is an artefact of multi-release JAR keying, not a limit of the carrier. The follow-up release closes the gap. A more substantive limitation: roundtrip equality is on the canonical-form string, not on a normalised semantic theory, so equivalent specifications written in different forms will appear distinct on the wire. This is the deliberate separation-of-concerns chosen in Section III; semantic normalisation is a separable layer.

VII. THREATS TO VALIDITY

a) Internal validity.: The 100% roundtrip-fidelity figure depends on the writer covering every method shape that ASM emits. Section V discusses the original 3% gap that arose when the writer’s lazy emission missed the abstract / interface case (only `visitEnd` fires for them). Adding `visitEnd` to the trigger set closed the gap on the libraries tested. The same risk pattern applies to method shapes specific to future JVM features — e.g., abstract record components, hidden classes (JEP 371) — which would need their own triggers if they exhibit different visit-call orderings. The mitigation is the same: a corpus-scale validation per JVM release.

The byte-overhead figure is measured under a deliberately bulky synthetic-spec workload (one annotation per parameter for every method). Real inferred specifications are typically

smaller per method, so the reported 11.76%–13.49% range should be read as an upper bound for the embedder’s contribution to JAR size. The 13.49% figure on Commons IO reflects its higher proportion of interfaces (each abstract method receives an embedded annotation just like a concrete method); a workload-aware embedder that elides annotations on abstract methods whose specs are pure pass-throughs would lower the figure further.

b) External validity.: The two libraries reported are mature, well-structured, and ship through Maven Central. Generalisation to less-tidy code (older artefacts, vendored dependencies, hand-tuned ProGuard-stripped JARs) will need separate validation. The negative test confirms that the consumer-side contract (load classes without the annotation type) holds at JDK 21; older JDKs (8, 11, 17) and ahead-of-time compilation paths (GraalVM `native-image`) are deferred.

The OpenAPI extension is validated by construction (it is a JSON Schema valid against OpenAPI 3.x’s extension rules). Empirical validation against a real microservice is part of the planned follow-up evaluation but does not appear in the present article.

c) Construct validity.: “Roundtrip fidelity” is operationalised as set-equality on (`internalName`, `methodName`, `descriptor`) keys. Two methods with identical signatures but different bytecode bodies (for example, the same constructor compiled with different optimisation levels) appear identical to the harness. This is the right construct for the embedder’s job — the embedder carries metadata, not bytecode — but a reader that needs to attach annotations to a specific compilation of a method would need additional disambiguation.

d) Conclusion validity.: The empirical claims are point measurements on two libraries; no statistical significance test is meaningful. The claims are robust in the sense that the gap-causing patterns are identified and fixable, and the absolute throughput numbers are well above the thresholds CI/CD workflows require, but a wider corpus (the protocol’s full ten libraries) is required before the claims generalise.

VIII. RELATED WORK

a) Specification languages.: JML [3], [4] provides the contract vocabulary we transport. Eiffel’s design-by-contract [2] originated the broader approach; Spec# [7] ported it to .NET. None of these languages address the distribution problem we tackle — they assume the consumer has source.

b) Specification inference.: The inference engine of [1] is the immediate predecessor of this work. Daikon [6] infers likely invariants from execution traces; Houdini and its descendants infer candidate annotations and discharge them through a verifier. PreInfer [8] uses dynamic symbolic execution. Jdoctor [9] extracts exception preconditions from Javadoc. None of these systems address what happens to the inferred specifications after they are produced; we provide the missing distribution layer.

c) *Annotation-based contracts.*: JSR-305 captured nullability annotations and was withdrawn before final standardisation; the surviving informal usage covers only nullability. The Checker Framework’s annotation vocabulary is type-system-shaped and does not generalise to the relational and quantified clauses JML supports. Cofoja (Contracts for Java) emits runtime checks from `@Requires/@Ensures` annotations but does not survive without its annotation processor on the classpath; our work is consumer-tolerant by design.

d) *Bytecode metadata transport.*: ASM [10] is the standard library for class-file rewriting and underpins our embedder. ByteBuddy and Javassist provide higher-level APIs over similar primitives. None of these libraries prescribe a metadata format for specifications — they are mechanism, not policy. JEP 181 (nest-mates) and JEP 371 (hidden classes) extend what the JVM accepts as class-file content but do not change the annotation channel.

e) *API description.*: OpenAPI 3.x is the standard for HTTP service descriptions; gRPC’s `.proto` files cover RPC. Pact [11] captures consumer-driven contracts as example interactions; it is example-based rather than logical, so it is a transport for examples rather than for specifications, and we specifically reject example-based encoding (Section III). The OpenAPI extension we propose is the smallest viable addition to an existing widely deployed format.

f) *Generating OpenAPI from source.*: Recent work on generating OpenAPI descriptions from Java bytecode and source code includes Respector, Prophet, and AutoOAS [12], which extract REST endpoint metadata from Spring Boot applications. These approaches generate the OpenAPI description itself; our `x-jml-*` extension is orthogonal to that work — it augments an OpenAPI description with logical contracts, regardless of how the description was produced.

g) *Mutation testing and test generation.*: The mutation-testing literature is cited in detail in [1]. The relationship is: that work measured what specifications do for LLM-generated tests, and the present work makes the specifications transportable to LLMs and other consumers that do not have the source.

IX. CONCLUSION

We asked whether automatically inferred specifications can be embedded in compiled Java artefacts and REST API descriptions with sufficient fidelity that downstream consumers can use them as if they had the source. We answered in the affirmative, with two specific constructions:

- 1) A repeatable runtime-retained Java annotation, `@JmlSpec`, written into class-file `RuntimeVisibleAnnotations` attributes by an ASM-based embedder. Roundtrip fidelity is 100.0% on three real-world libraries (Apache Commons Lang, Apache Commons IO, Guava — totalling 20,546 methods); bytecode-size overhead is 11.76–13.49%; embedding throughput is in the 10,000–31,000

methods/second range (medians 18,859–29,358 across 39 runs); reading throughput is in the 134,000–316,000 methods/second range (medians 162,002–275,834).

- 2) An OpenAPI 3.x extension family (`x-jml-requires`, `x-jml-ensures`, `x-jml-assignable`, `x-jml-signals`, `x-jml-invariant`) per-operation and per-schema, plus a sidecar JSON fallback for documents generated by frameworks. The extension is preserved by standard OpenAPI parsers without modification.

Our negative test confirms that classes embedded with `@JmlSpec` load correctly in JVMs that have no awareness of the annotation type — the embedder is consumer-tolerant by design. Together with the inference engine of [1], the path from source code to consumer-accessible contract is complete: the inferer produces specifications, the embedder ships them alongside compiled artefacts, and the OpenAPI extension carries them across service boundaries.

We see three threads of follow-up work. First, validation across the remaining seven libraries identified in our protocol (Apache Commons Math, jOOL, Vavr, jsoup, JFreeChart, JUnit, AssertJ). Second, integration with the LLM-based test generator of [1], which can now consume specifications directly from the JAR rather than requiring source. Third, an extension of the inference itself to compose specifications across method boundaries; the carrier built here is what makes that composition transportable to consumers.

The replication package, including the full source of the embedder/extractor pair, the OpenAPI extension JSON Schema, and the empirical validation harness, accompanies this paper.

DATA AVAILABILITY

A replication package containing the embedder/extractor source, the OpenAPI extension JSON Schema, raw measurement logs, and reproduction scripts is available at [https://github.com/\[anonymized\]/jml-inferer](https://github.com/[anonymized]/jml-inferer).

REFERENCES

- [1] B. Edmonds and M. Utting, “Do specifications help LLMs write better unit tests? an empirical study of specification-guided test generation with heuristically inferred JML,” 2026, manuscript under review at the Journal of Software: Evolution and Process. Replication package: <https://github.com/ziggyfish/jml-inferer>.
- [2] B. Meyer, “Applying “design by contract”,” *Computer*, vol. 25, no. 10, pp. 40–51, 1992.
- [3] G. T. Leavens, A. L. Baker, and C. Ruby, “Preliminary design of JML: A behavioral interface specification language for Java,” *ACM SIGSOFT Software Engineering Notes*, vol. 31, no. 3, pp. 1–38, 2006.
- [4] G. T. Leavens *et al.*, “JML reference manual,” <http://www.jmlspecs.org>, 2013, accessed: 2025-01-15.
- [5] D. R. Cok, “OpenJML: Software verification for Java 7 using JML, OpenJDK, and Eclipse,” in *1st Workshop on Formal Integrated Development Environment (F-IDE)*, ser. EPTCS, vol. 149, 2014, pp. 79–92.
- [6] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, “The Daikon system for dynamic detection of likely invariants,” *Science of Computer Programming*, vol. 69, no. 1–3, pp. 35–45, 2007.
- [7] M. Barnett and K. R. M. Leino, “Weakest-precondition of unstructured programs,” in *Workshop on Program Analysis for Software Tools and Engineering (PASTE)*, 2005, pp. 82–87.

- [8] A. Astorga, S. Srisakaokul, X. Xiao, and T. Xie, “PreInfer: Automatic inference of preconditions via symbolic analysis,” in *48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2018, pp. 678–689.
- [9] A. Blasi, A. Goffi, K. Kuznetsov, A. Gorla, M. D. Ernst, M. Pezzè, and S. D. Castellanos, “Translating code comments to procedure specifications,” in *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA ’18. ACM, Jul. 2018, pp. 242–253. [Online]. Available: <http://dx.doi.org/10.1145/3213846.3213872>
- [10] OW2 Consortium, “ASM: a Java bytecode manipulation and analysis framework,” <https://asm.ow2.io/>, 2024, accessed: 2026-05-19.
- [11] Pact Foundation, “Pact: consumer-driven contract testing,” <https://docs.pact.io/>, 2024, accessed: 2026-05-19.
- [12] A. Lercher, C. Macho, C. Bauer, and M. Pinzger, “Generating accurate OpenAPI descriptions from Java source code,” in *Proceedings of the 32nd IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2025, arXiv preprint: <https://arxiv.org/abs/2410.23873>.